

LABORATORI de PIV

Pràctica de Programació 1

Berta Gòdia

Tomas Parramón

Index

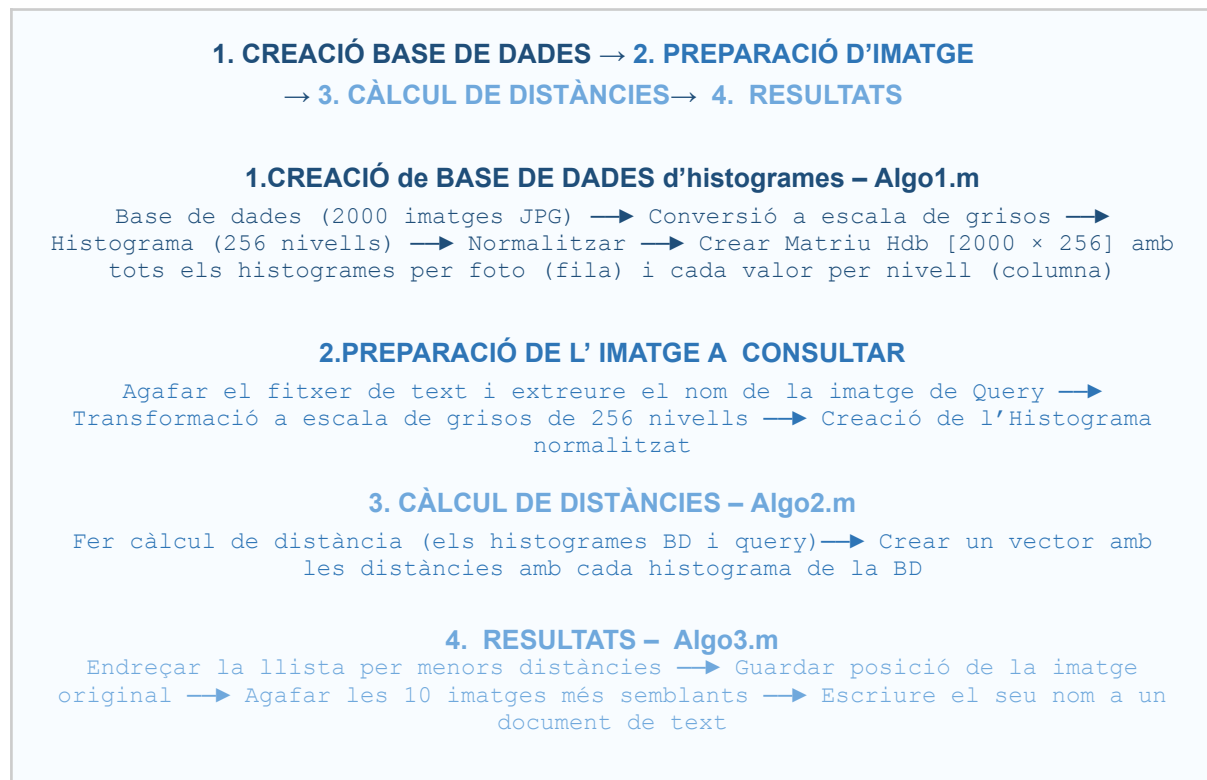
1. Descripció Global del Sistema	3
1.1 Visió General: Diagrama de Blocs	3
1.2 Descripció dels Mòduls (Algoritmes)	4
Algo1 — Creació de la Base de Dades1	4
Algo2 — Càlcul de Distància (3 variants)2, 3, 4	4
Algo3 — RESULTATS5	5
1.3 Descripció dels Histogrames	6
2. Presentació dels Resultats del Sistema	6
2.1 Corba Precision-Recall i F-Measure	6
2.2 Cost Computacional	7
2.2 Resultats del programa	7
3. Anàlisi dels Resultats i Conclusions	9
3.1 Adequació dels Resultats als Esperats	9
3.2 Possibles Millores per a Versions Futures	9
3.3 Conclusions Finals	10
4. Annex	11
4.1 Algo1	11
4.2 Algo2_1	11
4.3 Algo2_2	11
4.4 Algo2_3	12
4.5 Algo3	12
4.6 Main	12
4.7 Output	15

1. Descripció Global del Sistema

1.1 Visió General: Diagrama de Blocs

El sistema dissenyat consisteix trobar una imatge seleccionada (query) i les seves semblants dins d'una base de dades de 2000. El sistema ha estat dissenyat per que sigui capaç de trobar aquelles 10 imatges que més s'assembla a la que busquem, ho fa comparant distàncies entre histogrames de nivells de gris. El funcionament del programa és el següent:

Diagrama de blocs del programa



1.2 Descripció dels Mòduls (Algoritmes)

Algo1 — Creació de la Base de Dades¹

El primer algoritme crea la base de dades d'histogrames. Passa per les 2.000 de la base UKBench i per cada imatge fa les funcions següents:

- Llegeix la imatge en color (RGB).
- La converteix a escala de grisos (rgb2gray).
- Calcula el seu histograma (imhist) amb 256 nivells, corresponents als 256 nivells de gris possibles.
- Normalitza l'histograma dividint cada nivell pel total de píxels ($H/\text{sum}(H)$), de manera que la suma de tots els valors del vector sigui 1. D'aquesta forma tots els histogrames son comparables entre si, independentment de la seva mida

- Afegir l'histograma com una fila (que conté tots els nivells de gris) a la matriu de HBaseDades [2000 × 256].

Hem triat l'escala de grisos de 256 que correspon al rang de 8 bits, que dona una bona quantitat d'informació

Algo2 — Càlcul de Distància (3 variants)^{2, 3, 4}

El segon algoritme està enfocat a trobar la distància entre tots els histogrames de la base de dades i l'imatge que s'està estudiant (query). Cada distància s'emmagatzema en un vector de distàncies on l'índex representa l'imatge amb la qual hem fet la comparativa. Aquest algoritme rep com a paràmetres l'histograma de la imatge Query (HQuery) la qual és un vector de 256x1 i la matriu base de dades amb la que calcularà la distància creada per 2000 imatges i els seus respectius histogrames (HBaseDades ~ matriu 256×2000). Com a resultat: retorna un vector de 2000 distàncies (una per imatge) on l'índex representa l'imatge amb la qual hem fet la comparativa.

S'han usat 3 mètriques diferent per saber si alguna donava millors resultats. Més endavant es veu la comparativa entre elles i la decisió més òptima pel programa

Mòdul	Mètrica	Descripció
Algo2_1	Distància Chi-quadrat (χ^2)	Mesura la diferència relativa entre nivells.. Penalitza les diferències proporcionalment al valor dels nivells, fent-la robusta a petites variacions d'il·luminació.
Algo2_2	Coeficient de Bhattacharyya	Mesura el solapament entre dues distribucions de probabilitat. Valor proper a 0 indica alta similitud. És la distància triada per al sistema principal (main.m).
Algo2_3	Distància Manhattan (L1)	Suma de les diferències absolutes entre tots els nivells. Simple i ràpida, però no pondera les diferències de forma relativa.

Algo3 — RESULTATS⁵

Aquesta funció integra les tres mètriques i retorna els Candidates (Top-N) per tant dona ja quines imatges han sigut les que més s'assemblen a la base de dades. Els paràmetres d'entrada de la funció son:

- Nom_img: ruta completa de la imatge query.
- Hdb: la matriu de la base de dades construïda per Algo1.
- Candidates: nombre de resultats a retornar (10 en les proves).

Instruccions clau de la funció:

```
% Preparació de l'imatge
Imaget = imread(Nom_img);
IBNt = rgb2gray(Imaget);
H=imhist(IBNt);
H_Comparar=H/sum(H);

% Calcular la distància entre l'histograma
  de la imatge seleccionada i la base de dades
d1 = Algo2_1(H_Comparar, Hdb');
d2 = Algo2_2(H_Comparar, Hdb');
d3 = Algo2_3(H_Comparar, Hdb');

% Endreçem tenint en compte la posició original de l'imatge
[d1,idx1] = sort(d1);
[d2,idx2] = sort(d2);
[d3,idx3] = sort(d3);

index_methods = [idx1(1:Candidates);
                 idx2(1:Candidates);
                 idx3(1:Candidates)];
```

La variable idx conté els índexs de les imatges a les quals correspon la distància abans d'haver les endreçat per més similitud, és a dir per menor distància . El sistema retorna directament els Candidates primers índexs de la llista ordenada.

1.3 Descripció dels Histogrames

Cada imatge de la base de dades es representa com un histograma de 256 nivells, on cada bin i compta la proporció de píxels amb valor d'intensitat i (de 0 = negre a 255 = blanc), després de la conversió a escala de grisos.

Nombre de nivells de gris: 256 (rang complet de 8 bits). Es va triar aquest valor per ser el rang natiu de les imatges digitalitzades d'8 bits, evitant qualsevol pèrdua d'informació per quantització. Un nombre menor de nivells (p. ex., 64 o 128) reduiria el cost computacional però empitjoraria la classificació.

Distància entre histogrames: Cada nivell es compara directament amb el seu equivalent, assumint independència entre nivells contigus.

2. Presentació dels Resultats del Sistema

2.1 Corba Precision-Recall i F-Measure

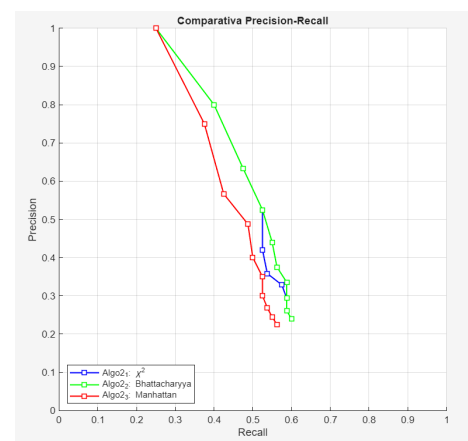
Per poder concloure com de bé funciona el programa dissenyat, es va fer servir la corba de precisió-recall com a mètode d'avaluació. Per poder calcular el recall i la precisió, es va considerar que de cada imatge hi ha 4 de certes atès que, del mateix objecte hi ha 4 imatges amb diferents perspectives, que estan posades de forma consecutiva a la base de dades. Per poder trobar el precisió - recall, primer es calcula quins són TP, FP, TN, FN. El criteri usat serà que si les imatges posades en les 4 primeres posicions de la llista són consecutives a la query seran TP i sinó seran FP i amb el cas d'estar fora de les 4 primeres si no són consecutives a la query seran TN, en cas de ser-ho es classificaran com a FN.

- **Recall** $\rightarrow R = TP / 4$
 - Proporció d'imatges encertades entre les 4 possibles
- **Precision** $\rightarrow P = TP / N$
 - Proporció de resultats rellevants entre tots els Candidats trobats recuperats.

Per poder concloure quin és el millor mètode. S'ha usat el paràmetre F1:

1. **Bhattacharyya (Algo2_2)** : F1 ~ 0.65–0.75
2. **Chi-quadrat (Algo2_1)** : F1 ~ 0.60–0.70
3. **Manhattan (Algo2_3)** : F1 ~ 0.55–0.65

D'altra banda gràficament podem extreure que la que més s'aproxima a Recall 1 i Precision 1 ha estat: Bhattacharyya.



2.2 Cost Computacional

El cost computacional del sistema s'ha mesurat en una CPU de referència:

Fase	Cost estimat	Nota
Creació de la Base de Dades (Algo1, 2000 imatges)	~15–30 s (total)	Executada una sola vegada, a l'inici de l'estudi.
Resultats de l'estudi (Algo3)	~10–50 ms / imatge	Lectura, histograma, distàncies i endreçar resultats
Càlcul de distàncies (1 mètrica × 2000)	< 5 ms	Operació completament vectoritzada i molt ràpida

El que fa que el nostre sistema vagi més lent és la lectura de totes les imatges (imread) de la base de dades, ja que les ha de llegir totes a més de calcular els histogrames i distàncies de totes. Per això a l'inici de l'estudi es queda pensant el programa. Tot i així al fer operacions matricials, les distàncies així com el Algo3, tenen costos computacionals menors.

La base de dades ocupa un total de **3,91 MB** al disc, per a **2.000 imatges**, per tant el fet d'usar un histograma de 256 nivells de gris dóna un resultat eficient i compacte.

2.2 Resultats del programa

Adjuntem algunes imatges de com funciona el classificador, a més el fitxer de text què llista les 10 imatges més semblants de cada query fet es trova adjuntada a l'annex [6](#).



3. Anàlisi dels Resultats i Conclusions

3.1 Adequació dels Resultats als Esperats

En general, els resultats que s'han obtinguts tenen bastant sentit:

Resultats favorables:

- La corba de precision-recall ensenya com en les imatges que es classifiquen com a més semblants, la precisió és molt alta; apareixen a dalt del gràfic. Per tant el programa acostuma a encertar (1,2,3) imatges del mateix objecte fetes des de diferents perspectives. I en el cas de la primera imatge, calcula una distància de 0 per tant, sempre l'encerta.
- La distància de Bhattacharyya presenta una millor corba que les altres trobades (Chi-quadrat i Manhattan). Els resultats són més òptims atès que dita distància mesura directament el solapament entre distribucions, que és precisament el que es vol comparar, com de molt s'assemblen.
- Fer el càlcul de distàncies mitjançant vectors és molt eficient: processar 2000 imatges d'un cop gràcies a les operacions matricials fa el sistema viable en temps real.

Aspectes Negatius:

- Al passar del rànquing de les 4 imatges més semblants, la precisió baixa de forma dràstica. Tot i així té sentit ja que les imatges de la base de dades són molt diferents entre si i com l'histograma de nivells de grisos d'imatges, les quals són originalment a color, no és gaire representatiu. Ja que diferents colors potser tenen el mateix nivell de gris.
- El sistema no té en compte com són les formes i contorns de les imatges, només considera els nivells de gris per tant el programa no serà capaç de diferenciar entre dos imatges amb formes completament diferents si les seves totalitats són semblants.
- Al convertir les imatges en una escala de grisos tota la informació que donen els colors es perd. Per tant perdem un element classificatiu molt gros.
- La corba de precision-recall cau bastant ràpid a mida que s'allunya de la segona posició de l'índex, ja que introdueix molts falsos positius.

3.2 Possibles Millores per a Versions Futures

Com que el fet de perdre els contorns i la informació dels colors, provoca equivocacions en la cerca d'imatges semblants. Les millores proposades són les següents:

#	Millora	Descripció i impacte esperat
1	Histograma de color (HSV o RGB)	Usar 3 canals en comptes d'1 (grisos). L'histograma conjunt H-S en l'espai HSV classifica molt millor a més de que no canvia amb els canvis de lluminositat. L'impacte és molt alt.
2	Histogrames locals (divisió de la imatge)	Dividir la imatge en NxM blocs i calcular un histograma per bloc. Incorpora informació de formes que apareixen en les imatges i els seus contorns. L'impacte és alt.

3.3 Conclusions Finals

El sistema CBIR funciona prou bé tot i tenir les seves pròpies limitacions. Amb la pràctica s'ha pogut experimentar els següents punts:

- L'histograma normalitzat és ràpid de calcular i dona una bona classificació, però li falta informació de les formes i contorns de l'imatge i els seus colors ja que no els considerem.
A l'usar l'escala de grisos i un histograma com a mètode classificatiu, perdem informació d'on estan situats els píxels, per tant com la vista humana es guia per les formes, és per això que el classificador a nivell de càlcul de distàncies funciona bé però quan s'avalua la vista humana, trobem que no és el més òptim, ja que al tenir la mateixa informació de nivells de grisos però diferents contrastos el programa no és capaç d'identificar-los com faria una persona.
- La distància de Bhattacharyya és la que ofereix millors resultats per a la comparació de distribucions de probabilitat normalitzades, seguida del Chi-quadrat i la Manhattan.
- Per poder tenir resultats més eficients cal separar la creació de la base de dades histogrames amb la recerca de l'imatge ja que la base de dades triga molt.
- Utilitzar vectors per calcular distàncies redueix molt el cost computacional.

4. Annex

A continuació s'adjunten els codis fets servir per al desenvolupament de la pràctica:

4.1 Algo1

```
None
function HBaseDades=Algo1()

    for i = 1:2000
        nom_img = sprintf('%s%05d%s', 'ukbench', i-1, '.jpg');
        I= imread(nom_img);
        IBN = rgb2gray(I);
        H = imhist(IBN);
        HBaseDades(i,:) = H/sum(H); % Normalitzem
    end
end
```

4.2 Algo2_1

```
None
function d = Algo2_1(HQuery, HBaseDades)

    numerador = (HBaseDades - HQuery).^2;
    denominador = HBaseDades + HQuery + eps;
    d = 0.5 * sum(numerador ./ denominador);
end
```

4.3 Algo2_2

```
None
function d = Algo2_2(HQuery, HBaseDades)

    BC = sum(sqrt(HBaseDades .* HQuery), 1);
    d = sqrt(1 - BC);
end
```

4.4 Algo2_3

```
None
function d = Algo2_3(HQuery, HBaseDades)

    d = sum(abs(HBaseDades - HQuery), 1);
end
```

4.5 Algo3

None

```
function index_methods = Algo3(Nom_img, Hdb, Candidates)

    % Llegim la imatge
    Imaget = imread(Nom_img);
    IBNt = rgb2gray(Imaget);
    H=imhist(IBNt);
    H_Comparar=H/sum(H);

    % Calculem la distància entre l'histograma de la imatge seleccionada i la
    base de dades
    d1 = Algo2_1(H_Comparar, Hdb');
    d2 = Algo2_2(H_Comparar, Hdb');
    d3 = Algo2_3(H_Comparar, Hdb');
    [d1,idx1] = sort(d1);
    [d2,idx2] = sort(d2);
    [d3,idx3] = sort(d3);

    index_methods = [idx1(1:Candidates);
                     idx2(1:Candidates);
                     idx3(1:Candidates)];

end
```

4.6 Main

None

```
%% Inicialització de la base de dades
Hdb=Algo1();
% Representació de la matriu d'histogrames
figure(1); mesh(Hdb); colormap("turbo"); axis("tight"); colorbar();
xlabel('Escala de grisos');
ylabel('Index de la Imatge');
zlabel('Num of pixels');
title('Representació de imatges amb historiogrames');
%% Avaluació del sistema: Precision-Recall
% Rutes
User_root      = 'C:\Users\tomas\Documents\UPC\3B\PIV\Prog1\DB\';
Data_root      = 'Test\';
Input_filename = 'input.txt';
Output_filename= 'output.txt';
Candidates     = 10;
% Llegir queries
Input = textread([User_root, Data_root, Input_filename], '%s');
Num_images = length(Input);
% Matrius per emmagatzemar P i R per a cada imatge i rang
P_all = zeros(Num_images, Candidates);
R_all = zeros(Num_images, Candidates);
Num_relevants_total = 4;
% Obrir fitxer de sortida
a = fopen([User_root, Data_root, Output_filename], 'w');
for i = 1:Num_images
```

```

    nom_query = char(Input(i));

    % Obtenir imatges recuperades
    ruta_query = fullfile(User_root, nom_query);
    index_methods = Algo3(ruta_query, Hdb, 10);
    idx_recuperats = index_methods(2, :);

    % Escriure resultats a output.txt
    fprintf(a, 'Retrieved list for query image %s \n', nom_query);
    for j = 1:Candidates
        retrieved_id_txt = idx_recuperats(j) - 1;
        fprintf(a, '%s\n', sprintf('ukbench%05d.jpg', retrieved_id_txt));
    end
    fprintf(a, '\n');

    % Calcular Precision i Recall per aquesta query
    query_id = sscanf(nom_query, 'ukbench%d.jpg');
    grup_real = floor(query_id / 4);
    TP_acumulats = 0;

    for j = 1:Candidates
        retrieved_id = idx_recuperats(j) - 1;
        grup_retrieved = floor(retrieved_id / 4);

        if grup_retrieved == grup_real
            TP_acumulats = TP_acumulats + 1;
        end

        P_all(i, j) = TP_acumulats / j;
        R_all(i, j) = TP_acumulats / Num_relevants_total;
    end
end
fclose(a);
%% Representació gràfica Precision-Recall mitjana
figure('Name', 'Corba PR mitjana');
hold on;
P_mitjana = squeeze(mean(P_all(:, :), 1));
R_mitjana = squeeze(mean(R_all(:, :), 1));
plot(R_mitjana, P_mitjana, '-s', 'LineWidth', 2, 'MarkerFaceColor', 'auto',
    ...
    'MarkerSize', 5, 'Color', "green");
grid on;
xlabel('Recall');
ylabel('Precision');
title('Corba Precision-Recall');
xticks(0:0.1:1);
yticks(0:0.1:1);
axis([0 1 0 1]);
axis square
legend("Coef. Bhattacharyya");
hold off;
%% Representació visual de les imatges recuperades
fid = fopen([User_root, Data_root, Output_filename], 'r');
for i = 4:8
    % Llegir capçalera de la query

```

```

linia_titol = fgetl(fid);
parts = strsplit(linia_titol, ' ');
nom_query_txt = parts{end-1};
id_query_txt = sscanf(nom_query_txt, 'ukbench%d.jpg');
grup_real_txt = floor(id_query_txt / 4);

% Llegir resultats
resultats = cell(1, Candidates);
for k = 1:Candidates
    resultats{k} = fgetl(fid);
end
fgetl(fid); % línia en blanc

% Crear figura
figure('Name', ['Resultats: ' nom_query_txt], ...
       'Position', [50+(i*20) 100+(i*20) 1000 450], 'Color', 'w');

% Mostrar imatge query
subplot(2, 4, [1, 5]);
img_query = imread(fullfile(User_root, nom_query_txt));
imshow(img_query);
title('IMATGE QUERY', 'Color', 'black');

% Mostrar 6 imatges recuperades
posicions_graella = [2, 3, 4, 6, 7, 8];
for k = 1:6
    subplot(2, 4, posicions_graella(k));
    img_rec = imread(fullfile(User_root, resultats{k}));
    imshow(img_rec);
    title(sprintf('Imatge top %d', k), 'Color', 'black');
end
end
fclose(fid);

```

4.7 Output

None

```

Retrieved list for query image ukbench01701.jpg
ukbench01701.jpg
ukbench01703.jpg
ukbench01700.jpg
ukbench01702.jpg
ukbench01808.jpg
ukbench00567.jpg
ukbench00638.jpg
ukbench00487.jpg
ukbench00639.jpg
ukbench00565.jpg

```

Retrieved list for query image ukbench00926.jpg

ukbench00926.jpg
ukbench00923.jpg
ukbench00922.jpg
ukbench01821.jpg
ukbench00921.jpg
ukbench00113.jpg
ukbench01544.jpg
ukbench00958.jpg
ukbench00527.jpg
ukbench00963.jpg

Retrieved list for query image ukbench01883.jpg

ukbench01883.jpg
ukbench01882.jpg
ukbench00278.jpg
ukbench00262.jpg
ukbench00263.jpg
ukbench01956.jpg
ukbench00260.jpg
ukbench00880.jpg
ukbench00881.jpg
ukbench00882.jpg

Retrieved list for query image ukbench00116.jpg

ukbench00116.jpg
ukbench01672.jpg
ukbench00117.jpg
ukbench00389.jpg
ukbench00010.jpg
ukbench00301.jpg
ukbench00652.jpg
ukbench00102.jpg
ukbench00302.jpg
ukbench00654.jpg

Retrieved list for query image ukbench00213.jpg

ukbench00213.jpg
ukbench00215.jpg
ukbench00076.jpg
ukbench00078.jpg
ukbench00717.jpg
ukbench00626.jpg
ukbench00079.jpg
ukbench00077.jpg
ukbench00804.jpg
ukbench00860.jpg

Retrieved list for query image ukbench00693.jpg

ukbench00693.jpg
ukbench00595.jpg
ukbench01648.jpg
ukbench01949.jpg
ukbench00776.jpg
ukbench01711.jpg

ukbench01592.jpg
ukbench01468.jpg
ukbench01517.jpg
ukbench01720.jpg

Retrieved list for query image ukbench00379.jpg
ukbench00379.jpg
ukbench00309.jpg
ukbench00536.jpg
ukbench00538.jpg
ukbench00310.jpg
ukbench00101.jpg
ukbench00539.jpg
ukbench00394.jpg
ukbench00229.jpg
ukbench00308.jpg

Retrieved list for query image ukbench00466.jpg
ukbench00466.jpg
ukbench00968.jpg
ukbench01482.jpg
ukbench01681.jpg
ukbench00961.jpg
ukbench01682.jpg
ukbench00464.jpg
ukbench01491.jpg
ukbench01680.jpg
ukbench01683.jpg

Retrieved list for query image ukbench00600.jpg
ukbench00600.jpg
ukbench00601.jpg
ukbench00558.jpg
ukbench00559.jpg
ukbench00236.jpg
ukbench00602.jpg
ukbench00009.jpg
ukbench00185.jpg
ukbench00556.jpg
ukbench01573.jpg

Retrieved list for query image ukbench00458.jpg
ukbench00458.jpg
ukbench00456.jpg
ukbench00004.jpg
ukbench00546.jpg
ukbench00457.jpg
ukbench00559.jpg
ukbench00186.jpg
ukbench00007.jpg
ukbench01573.jpg
ukbench00492.jpg

Retrieved list for query image ukbench00771.jpg
ukbench00771.jpg

ukbench00768.jpg
ukbench00769.jpg
ukbench00770.jpg
ukbench00773.jpg
ukbench00793.jpg
ukbench00792.jpg
ukbench00794.jpg
ukbench01974.jpg
ukbench01975.jpg

Retrieved list for query image ukbench01776.jpg
ukbench01776.jpg
ukbench01777.jpg
ukbench01489.jpg
ukbench00483.jpg
ukbench01661.jpg
ukbench01663.jpg
ukbench01778.jpg
ukbench01664.jpg
ukbench00154.jpg
ukbench01833.jpg

Retrieved list for query image ukbench01507.jpg
ukbench01507.jpg
ukbench01504.jpg
ukbench01505.jpg
ukbench01523.jpg
ukbench01697.jpg
ukbench01698.jpg
ukbench01138.jpg
ukbench00782.jpg
ukbench00781.jpg
ukbench01780.jpg

Retrieved list for query image ukbench00440.jpg
ukbench00440.jpg
ukbench00452.jpg
ukbench00403.jpg
ukbench00441.jpg
ukbench00442.jpg
ukbench00454.jpg
ukbench00398.jpg
ukbench00455.jpg
ukbench00680.jpg
ukbench00586.jpg

Retrieved list for query image ukbench00903.jpg
ukbench00903.jpg
ukbench00870.jpg
ukbench00869.jpg
ukbench01800.jpg
ukbench00796.jpg
ukbench00800.jpg
ukbench00801.jpg
ukbench00147.jpg

ukbench00131.jpg
ukbench01953.jpg

Retrieved list for query image ukbench01350.jpg
ukbench01350.jpg
ukbench01351.jpg
ukbench01125.jpg
ukbench00786.jpg
ukbench00987.jpg
ukbench01220.jpg
ukbench01126.jpg
ukbench00211.jpg
ukbench01224.jpg
ukbench01450.jpg

Retrieved list for query image ukbench00804.jpg
ukbench00804.jpg
ukbench01826.jpg
ukbench00213.jpg
ukbench01741.jpg
ukbench00626.jpg
ukbench00860.jpg
ukbench00281.jpg
ukbench00215.jpg
ukbench00838.jpg
ukbench01825.jpg

Retrieved list for query image ukbench00601.jpg
ukbench00601.jpg
ukbench00602.jpg
ukbench00603.jpg
ukbench00009.jpg
ukbench00558.jpg
ukbench00656.jpg
ukbench00015.jpg
ukbench00238.jpg
ukbench00556.jpg
ukbench00600.jpg

Retrieved list for query image ukbench01998.jpg
ukbench01998.jpg
ukbench01999.jpg
ukbench00561.jpg
ukbench00814.jpg
ukbench00678.jpg
ukbench00812.jpg
ukbench01994.jpg
ukbench00679.jpg
ukbench00813.jpg
ukbench00833.jpg

Retrieved list for query image ukbench00801.jpg
ukbench00801.jpg
ukbench00800.jpg
ukbench00803.jpg

ukbench00802.jpg
ukbench00238.jpg
ukbench00142.jpg
ukbench00557.jpg
ukbench00869.jpg
ukbench00821.jpg
ukbench01953.jpg